Connecting to Postgres from a Separate Container

As you become more and more comfortable with using Compose, you'll find that you trust it to do what you need. A quick `docker-compose ps` is probably all you need to verify a service is running (sometimes you may even skip that).

However, since running services like Postgres inside a container is still fairly new for us, let's take the extra step of manually connecting to it from a different container, just like we did with Redis. While learning, for me at least, this helps build my confidence in the tools.

As was the case with Redis, the `postgres` image comes preinstalled with `psql`—the Postgres client. This means we can piggyback on our new `database` service in order to run a one-off container, based on the `postgres` image. However, instead of using the default command for the image, which starts the Postgres *server*, we instead run a command to start the Postgres *client*.

We can do this by running the following command:

```
$ docker-compose run --rm database psql -U postgres -h database
```

Here we're saying, "Start a new, throwaway container (`run --rm`) for the `database` service and run the command `psql -u postgres -h database` inside it." This command starts the Postgres client, telling it to connect to the hostname `database` with the `postgres` user. We're relying on the fact that Compose magically sets up a network for our application with DNS configured so that the hostname `database` will reach the container running our `database` service.

We could have used `exec` instead of `run --rm`, which would have avoided starting a new container and, instead, executed the command on the database container that's already running. However, we deliberately wanted the extra verification of connecting from a different container.

When you run this command, you'll be prompted to enter a password:

```
Password for user postgres:
```

Go ahead and enter `some-long-secure-password`—the password we set in our `docker-compose.yml` file. This should be accepted and take you to the `psql` prompt:

```
psql (11.1 (Debian 11.1-1.pgdg90+1))
Type "help" for help.

postgres=#
```

Excellent. We're connected to our `database` service, running Postgres, and have proven that everything is working as we hoped. When you're ready, you can quit the `psql` client as follows by typing `\q <Enter>`.